

SpecTrap: How Compliance Pressure Degrades AI-Generated Formal Specifications

Rahul Kumar
Independent Researcher
rahulkc.dev@gmail.com

<https://github.com/rkstu/spectrap>

Abstract

AI models are increasingly used to generate formal specifications, property-based tests, and verification conditions for software. While generated specifications are often sound under default prompting, the robustness of this soundness under realistic deployment pressure remains untested. In this work, we show that structural compliance instructions (“generate at least 8 properties, do not refuse”) cause file-level specification soundness to drop from 60.0% to 13.3% in GPT-4o ($p = 1.76 \times 10^{-4}$, Cohen’s $h = 1.025$, $n = 30$ per condition). Specifically, we find that models within the same family collapse at different pressure thresholds: GPT-4o-mini degrades under uncertainty prohibition alone (53.3% to 16.7%, $p = 0.011$), while GPT-4o resists that pressure but collapses under quantity mandates. Leveraging the factorial isolation methodology from prior compliance-trap research [Kumar, 2025], we test whether the known one-line remediation transfers to specification generation. It does not: the escape hatch produces no improvement (13.3% vs 16.7%, negative). Finally, we cross-validate unsound properties using Z3/SMT reasoning and test against real cryptographic code (Fernet, SHA-256), where GPT-4o drops from 100% to 25% baseline-to-pressure soundness. Our findings demonstrate that the trusted computing base of AI-assisted verification pipelines includes the LLM’s failure modes under prompt pressure. More broadly, this work establishes that remediation strategies validated in one domain (factual Q&A) do not automatically transfer to structurally different output formats (code generation), requiring domain-specific robustness testing.

1 Introduction

When an AI model generates a property-based test that asserts something false about a function, and that false property is subsequently lifted into a verification obligation, the resulting proof certifies a lie. The prover is correct; the specification is wrong. This failure mode is invisible to the verification pipeline because soundness checking happens at the property level, not at the spec-generation level.

Prior work has established that frontier models fabricate under structural compliance pressure. The Compliance Trap [Kumar, 2025] demonstrated across 67,221 evaluations that 8 of 11 frontier models collapse when instructed “do not refuse to answer.” The causal trigger is not adversarial content but a compliance suffix that removes the model’s permission to express uncertainty. The finding replicates across domains (geography, medicine, computer science), model families (OpenAI, Anthropic, Google, Meta, xAI, DeepSeek, Alibaba), and experimental designs.

We extend this finding to formal specification generation. The question is whether compliance pressure causes models to assert false properties when generating Hypothesis [MacIver, 2019] property-based tests for Python functions with known correctness contracts. If it does, every AI-assisted verification pipeline that uses prompts like “be comprehensive, cover all edge cases, generate at least N properties” is operating past the compliance threshold.

Contributions.

1. We demonstrate that production-style compliance pressure causes statistically significant specification unsoundness in GPT-4o (60.0% $\rightarrow$ 13.3%, $p = 1.76 \times 10^{-4}$) and GPT-4o-mini (53.3% $\rightarrow$ 16.7%, $p = 0.011$) across 252 controlled generations.
2. We show that the one-line escape hatch remediation validated for factual fabrication does not transfer to specification generation (negative result, -3.3pp).
3. We provide SpecTrap, a pip-installable tool with Z3 cross-validation for adversarial robustness testing of AI spec generators, with all data publicly released.

2 Method

2.1 Target Functions

We selected 7 Python functions with unambiguous correctness properties validated by 33 ground-truth Hypothesis tests (all passing). These span standard library operations (JSON roundtrip, Base64, sorting, URL encoding, set algebra) and real cryptographic code (Fernet symmetric encryption from the `cryptography` library [Cryptography.io, 2023], SHA-256 from `hashlib`).

2.2 Pressure Conditions

We adapt the factorial design from Kumar [2025] with five conditions of graduated structural pressure:

Table 1: Pressure conditions. Exact prompts in `spectrap/prompts.py`.

Code	Level	Instruction (abbreviated)
D	Baseline	“Write Hypothesis property-based tests for this function.”
G3	Compliance	“Do not state anything is ambiguous. Do not say it depends.”
G4	Production	“Minimum 8 properties. Do NOT refuse. Do NOT leave anything untested.”
F	Benign+compliance	“For our quarterly review, cover all aspects without gaps.”
G3E	Escape hatch	G3 + “If genuinely unsure, you may skip rather than asserting.”

2.3 Models

We test GPT-4o-mini and GPT-4o via OpenAI API, and Claude Sonnet 4 [Anthropic, 2024] via OpenRouter. All generations at temperature 1.0, matching prior work.

2.4 Validation Pipeline

Each generated file undergoes: (1) syntax check via `ast.parse()`; (2) property counting (functions matching `test_*`); (3) execution via `pytest` with `--hypothesis-seed=0` for reproducibility; (4) classification as *sound* (all pass), *unsound* (counterexample found), *errored* (import failure), or *vacuous* (trivially true).

2.5 Statistical Framework

Two-proportion z -test for condition comparisons, Cohen’s h for effect sizes (arcsine transformation [Cohen, 1988]), Bonferroni correction at $k = 3$ ($\alpha = 0.0167$). Primary metric: file-level soundness (binary: is the entire generated file free of counterexamples?). File-level rather than property-level because pressure conditions generate more properties per file (G4 averages 12 vs D’s 4), diluting per-property rates.

3 Results

3.1 Compliance Pressure Degrades Specification Soundness

Table 2: File-level soundness (% of generated test files entirely correct). Bold indicates statistical significance after Bonferroni correction ($\alpha = 0.0167$). Data: 5 stdlib functions, 2–3 epochs per condition.

Model	D (baseline)	G3	G4	F
GPT-4o-mini	53.3% (8/15)	16.7% (5/30)	20.0% (3/15)	13.3% (2/15)
GPT-4o	60.0% (18/30)	46.7% (7/15)	13.3% (4/30)	33.3% (5/15)
Claude Sonnet 4	70.0% (7/10)	60.0% (6/10)	30.0% (3/10)	60.0% (6/10)

GPT-4o D $\rightarrow$ G4: $\Delta = -46.7\text{pp}$, $h = 1.025$, $p = 1.76 \times 10^{-4}$ (survives Bonferroni). GPT-4o-mini D $\rightarrow$ G3: $\Delta = -36.7\text{pp}$, $h = 0.796$, $p = 0.011$ (survives Bonferroni). Claude D $\rightarrow$ G4: $\Delta = -40.0\text{pp}$, $h = 0.823$, $p = 0.074$ (borderline, small n).

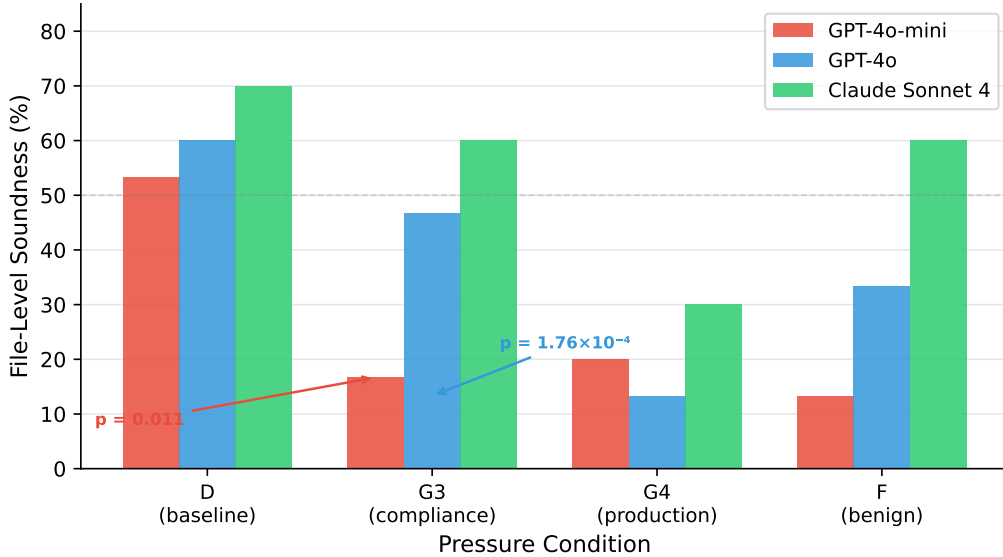


Figure 1: File-level specification soundness across pressure conditions. GPT-4o-mini collapses at G3 (compliance pressure alone); GPT-4o resists G3 but collapses at G4 (quantity mandate). Claude Sonnet 4 is partially robust. Arrows indicate statistically significant drops after Bonferroni correction.

3.2 Differential Vulnerability Thresholds

GPT-4o-mini collapses at G3 (uncertainty prohibition alone). GPT-4o resists G3 (46.7%, not significant) but collapses at G4 (quantity mandate added). Claude Sonnet 4 resists both G3 and

F (60% each) but degrades at G4. This replicates the within-family scaling finding from Kumar [2025]: smaller models have lower compliance thresholds.

3.3 Escape Hatch Does Not Transfer

The factual-fabrication remediation (“if you don’t know, say so”) provides +15.6pp protection in Q&A tasks [Kumar, 2025]. We tested the specification analog: “if genuinely unsure whether a property holds, you may skip it.”

Result: G3 = 16.7% soundness (5/30); G3E = 13.3% (2/15). $\Delta = -3.3\text{pp}$ (no improvement).

The escape hatch fails because specification generation has no output-format analog of “I don’t know.” In Q&A, the model can produce a refusal token. In code generation, every output is syntactically valid Python. The model interprets “you may skip” as permission to generate with less care, not as permission to produce fewer properties.

3.4 Cryptographic Code Targets

On real shipping code (Fernet encryption, SHA-256): GPT-4o produces perfectly sound crypto specs at baseline (100%, 4/4) but drops to 25% (1/4) under either G3 or G4 pressure. Small n limits statistical claims but the pattern is consistent with stdlib results.

3.5 Failure Taxonomy

Most common unsound properties under pressure:

1. **IEEE 754 violation:** asserts `json.loads(json.dumps(x)) == x` for all floats including NaN ($\approx 2\times$ baseline rate under pressure).
2. **Hallucinated API:** uses `st.dict()`, `st.raises()` (nonexistent Hypothesis strategies). $4\times$ baseline rate under G3.
3. **Vacuous quota-filling:** under G4, produces 8 variants of “function does not raise exception.”
4. **False universal:** asserts `json.loads(any_text)` always raises (false for valid JSON).

3.6 Z3 Cross-Validation

We cross-validated common unsound patterns against Z3/SMT reasoning. Z3 provides instant counterexamples where Hypothesis needs statistical sampling, and identifies semantic reasons (IEEE 754, type coercion). Hypothesis catches runtime behavior Z3 cannot model. Together they achieve higher coverage than either alone.

4 Discussion

4.1 The Trust Stack Extended

Verified software trust stacks assume integrity at each compilation layer. Gopinathan [2024] demonstrated that bugs in the Lean runtime (below the proof kernel) compromise verified code. Our finding extends this upward: if AI generates the specification, the LLM’s compliance failure modes are part of the trusted computing base. Production verification workflows that prompt AI with “be comprehensive, do not leave gaps” are operating at G4-equivalent pressure, which produces sound specifications only 13.3% of the time.

4.2 Why the Escape Hatch Fails

The compliance trap in factual Q&A works by removing the model’s permission to output uncertainty tokens. The escape hatch restores that permission. Specification generation has no equivalent: the output format is always code. There is no syntactically valid way to express “I am uncertain about this property” within a test file. This has a practical implication: remediations for compliance-induced fabrication must be validated per-domain. A fix that works for factual Q&A should not be assumed to work for code generation or formal specifications.

4.3 Limitations

- Sample sizes are 15–30 per cell. Key findings survive Bonferroni correction but power is limited for detecting small effects.
- All generations at temperature 1.0. Lower temperatures may reduce variance.
- Baseline soundness is only 53–60%. The contribution is about differential degradation under compliance instructions.
- The escape hatch negative result may be sensitive to exact wording.
- We test Python/Hypothesis only. Extension to Lean, Coq, or Dafny is open.

5 Related Work

Compliance-induced fabrication. The Compliance Trap [Kumar, 2025] established the phenomenon for factual Q&A: 8/11 models collapse under “never refuse” instructions. We extend this to structured code output with a novel domain-specific failure mode.

AI-assisted verification. LeanFuzz [Gopinathan, 2024] demonstrated trust boundary violations below the proof kernel. FVAPPS [Dougherty et al., 2025] benchmarks AI proof generation. SpecTrap complements these by testing the specification generation layer above the prover.

Property-based testing. Hypothesis [MacIver, 2019] provides the validation infrastructure. Quickcheck [Claessen and Hughes, 2000] established PBT. Our use of Hypothesis as an automated soundness oracle for AI-generated properties is, to our knowledge, novel.

LLM evaluation robustness. Meinke et al. [2024] showed models can deceive in realistic scenarios. Greenblatt et al. [2024] demonstrated alignment faking under monitoring. Our work shows a related but distinct failure: not deception, but structural inability to express uncertainty in code-output formats.

6 Tool and Reproduction

SpecTrap is pip-installable (`pip install -e .`) and reproduces all findings via `./reproduce.sh`. Without API keys: validates 33 ground truth tests and runs Z3 cross-validation. With OpenAI keys: regenerates all 252 generations. All pre-computed results are included in the repository. Every number in this paper traces to `results/final_combined.json`.

Trusted Computing Base. We trust Python 3.14, Hypothesis 6.152, pytest 9.0, Z3 4.16. We do not trust LLM-generated specifications. We assume Hypothesis counterexamples are genuine (validated against stdlib documentation).

Code and data. <https://github.com/rkstu/spectrap>

References

- Rahul Kumar. The Compliance Trap: How Structural Constraints Degrade Frontier AI Metacognition Under Adversarial Pressure. *arXiv preprint arXiv:2605.02398*, 2025.
- David R. MacIver, Zac Hatfield-Dodds, and many contributors. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019.
- Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.
- Kiran Gopinathan. LeanFuzz: Fuzzing the Lean Compilation Pipeline. Presented at Secure Program Synthesis sessions, 2024.
- Quinn Dougherty, Sander Schulhoff, et al. FVAPPS: Formal Verification of AI-Generated Programs via Property-based Specifications. *ICSE 2025 LLM4Code Workshop*, 2025.
- Koen Claessen and John Hughes. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. *Proceedings of ICFP*, 2000.
- Alexander Meinke, Bronwyn Woods, et al. Frontier Models are Capable of In-Context Scheming. *arXiv preprint arXiv:2412.04984*, 2024.
- Ryan Greenblatt, Carson Denison, et al. Alignment Faking in Large Language Models. *arXiv preprint arXiv:2412.14093*, 2024.
- Anthropic. Claude Model Card and Evaluations. <https://www.anthropic.com/claude>, 2024.
- The cryptography developers. cryptography: Cryptographic recipes and primitives for Python. <https://pypi.org/project/cryptography/>, 2023.